

Основы работы с пользовательскими функциями в JavaScript

Вы уже знаете многие стандартные функции JavaScript, например `substr()`, `reverse()`, `split()`. Их использование очень удобно - чтобы написать свои аналоги на JavaScript, потребовалось бы много времени и сил.

Однако, с помощью стандартных функций невозможно сделать все, что нам требуется. На помощь приходит такой механизм JavaScript, как **функции пользователя**. С их помощью мы можем создавать свои функции, принцип работы которых аналогичен стандартным функциям JavaScript.

Зачем нужны пользовательские функции?

Очень часто при программировании возникает такая ситуация: некоторые участки кода повторяются несколько раз в разных местах. Пользовательские функции нужны для того, чтобы убрать дублирование кода.

Дублирование плохо тем, что если вам потребуется что-то поменять в коде - это придется сделать во многих местах. При этом обязательно в каком-нибудь месте вы это сделать забудете. Практика копирования участков кода и вставки в другое место - очень **плохая практика** (очень часто ей грешат новички).

Кроме того, функции скрывают внутри себя какой-то код, о котором нам не приходится задумываться. Например, у нас есть функция `saveUser()`, которая сохраняет пользователя в базу данных. Мы можем просто вызывать ее, не задумываясь о том, какой код выполняется внутри функции. Это очень удобно. В программировании это называется *инкапсуляцией*.

Синтаксис функций пользователя

Функция создается с помощью команды **function**. Далее через пробел следует *имя функции* и *круглые скобки*. Круглые скобки могут быть **пустыми**, либо в них могут быть указаны **параметры**, которые принимает функция. Параметры - это обычные переменные JavaScript.

Сколько может быть параметров: один, несколько (в этом случае они указываются через запятую), ни одного (в этом случае круглые скобки все равно нужны, хоть они и будут пустыми).

// 'func' - это имя функции, ей передается один параметр param:

```
function func(param) {  
  
}
```

```
//Передаем функции два параметра - param1 и param2:
```

```
function func(param1, param2) {  
  
}
```

```
//Вообще не передаем никаких параметров:
```

```
function func() {  
  
}
```

Обратите внимание на фигурные скобки в примерах: в них мы пишем **код**, который выполняет функция. Общий синтаксис функции выглядит так:

```
function имя_функции(здесь перечисляются параметры через запятую) {  
    Здесь написаны команды, которые выполняет функция.  
}
```

Как вызвать функцию в коде?

Саму функцию мы можем вызвать в любом месте нашего JavaScript документа (даже *до ее определения*). Функция вызывается по ее **имени**. При этом круглые скобки после ее имени обязательны, даже если они пустые. Смотрите пример:

```
/*  
    Мы написали функцию,  
    которая выводит на экран 'Привет, мир!'.  
    Назовем ее 'hello':  
*/  
function hello() {  
    /*  
        В этом месте ничего не выведется само по себе,  
        а выведется только при вызове функции.  
    */  
    alert('Привет, мир!');  
}  
  
/*  
    Теперь мы хотим вызвать нашу функцию,  
    чтобы она вывела на экран свою фразу.  
    Обратимся к ней по имени:  
*/  
hello(); //Вот и все! Функция выведет на экран фразу 'Привет, мир!'.
```

```
/*
    Мы можем вывести нашу фразу несколько раз -
    для этого достаточно написать функцию
    не один раз, а несколько:
*/
hello();
hello();
hello();
```

Подробнее о параметрах

В наших примерах мы вызывали функцию **'hello()'** без параметров.

Давайте теперь попробуем ввести параметр, который будет задавать текст, выводимый нашей функцией:

```
//Зададим нашу функцию:
function hello(text) { //укажем здесь параметр text
    //Выведем на экран значение переменной text:
    alert(text);
}
```

//Теперь вызовем нашу функцию:

```
hello('Привет, Земляне!'); //она выведет именно ту фразу, которую мы ей передали
```

Обратите внимание на переменную **text** в нашей функции: в ней появляется то, что мы передали в круглых скобках при вызове функции.

Как JavaScript знает, что текст 'Привет, Земляне!' нужно положить в переменную **text**? Мы сказали ему об этом, когда создавали нашу функцию, вот тут: **'function hello(text)'**.

Если мы зададим несколько параметров - то каждый из них будет лежать в своей переменной внутри функции.

Инструкция return

Чаще всего функция должна не выводить что-то через alert на экран, а *возвращать*.

Сделаем теперь так, чтобы функция не выводила что-то на экран, а **возвращала текст** скрипту, чтобы он смог записать этот текст в переменную и как-то обработать его дальше:

```
//Зададим функцию:
function hello(text) {
    /*
        Укажем функции с помощью инструкции 'return',
        что мы хотим, чтобы она ВЕРНУЛА текст, а не вывела на экран:
    */
}
```

```
        return text;
    }
}
```

//Теперь вызовем нашу функцию и запишем значение в переменную message:

```
var message = hello('Привет, Земляне!'); //пока вывода на экран нет
```

//И теперь в переменной message лежит 'Привет, Земляне!':

```
alert(message); //убедимся в этом
```

//Можно не вводить промежуточную переменную message, а сделать так:

```
alert(hello('Привет, Земляне!'));
```

В принципе, практической пользы от того, что мы сделали - никакой, так как функция вернула нам то, что мы ей передали. Модернизируем наш пример так, чтобы функция принимала имя человека, а выводила строку 'Привет, %имя_человека_переданное_функции%!':

//Зададим функцию:

```
function hello(name) { //укажем здесь параметр name, в котором будет лежать имя человека
```

```
    //Введем переменную text, в которую запишем всю фразу:
```

```
    var text = 'Привет, '.name.'!';
```

```
    /*
```

```
        Укажем функции с помощью инструкции 'return',
```

```
        что мы хотим, чтобы она ВЕРНУЛА содержимое переменной text:
```

```
    */
```

```
    return text;
```

```
}
```

//Теперь вызовем нашу функцию и запишем значение в переменную message:

```
var message = hello('Дима');
```

//И теперь в переменной text лежит 'Привет, Дима!':

```
alert(message); //убедимся в этом
```

//Поздороваемся сразу с группой людей из трех человек:

```
alert(hello('Вася').' '.hello('Петя').' '.hello('Коля'));
```

Частая ошибка с return

После того, как выполнится инструкция **return** – функция закончит свою работу. То есть: *после выполнения return больше никакой код не выполнится.*

Это не значит, что в функции должен быть **один return**. Но выполнится только один из них. Такое поведение функций является причиной многочисленных трудноуловимых ошибок.

Смотрите пример:

```
function func(param){
    /*
        Если переменная param имеет значение true, то вернем 'Верно!'.
        Напоминаю о том, что конструкция if(param) эквивалентна if(param ===
true)!
    */
    if (param) return 'Верно!';
    /*
        Далее новичок в JavaScript хочет проделать еще какие-то операции,
        но если переменная param имеет значение true – сработает инструкция r
eturn,
        и код ниже работать не будет!

        Напротив, если param === false – инструкция return не выполнится
        и код дальше будет работать!
    */
    alert('Привет, мир!');
}
//Осознайте это и не совершайте ошибок
```

Приемы работы с return

Существуют некоторые приемы работы с **return**, упрощающие код. Как сделать проще всего:

```
function func(param) {
    /*
        Что делает код:
        если param имеет значение true – то в переменную result запишем 'Вер
но!',
        иначе 'Неверно!':
    */
    if (param) {
```

```

        var result = 'Верно!'
    } else {
        var result = 'Неверно!';
    }

    //Вернем содержимое переменной result:
    return result;
}

```

Теперь упростим нашу функцию, используя прием работы с return:

```

function func(param) {
    /*
        Что делает код:
        если param имеет значение true – вернет 'Верно!',
        иначе вернет 'Неверно!'.
    */
    if (param) {
        return 'Верно!'
    } else {
        return 'Неверно!';
    }
}

```

Обратите внимание на то, насколько упростился код. Плюсом также является то, что мы убрали лишнюю переменную **result**.

Рассматриваем примеры

Пусть у нас есть функция, которая выводит на экран квадрат переданного числа:

```

function func(num) {
    alert(num * num);
}

```

`func(3);` // выведет 9

Пусть мы хотим не выводить значение на экран, а записать в какую-нибудь переменную, вот так:

`let result = func(3);` // в переменной result теперь 9

Для этого в JavaScript существует специальная инструкция **return**, которая позволяет указать значение, которое *возвращает* функция.

Под словом *возвращает* понимают то значение, которое запишется в переменную, если ей присвоить вызванную функцию.

Итак, давайте перепишем нашу функцию так, чтобы она не выводила результат на экран, а *возвращала* его в переменную:

```
function func(num) {  
    return num * num;  
}
```

```
let result = func(3); // в переменной result теперь 9
```

После того, как данные записаны в переменную, их можно, например, вывести на экран:

```
function func(num) {  
    return num * num;  
}
```

```
let result = func(3);  
alert(result); // выведет 9
```

А можно сначала как-то изменить эти данные, а затем вывести их на экран:

```
function func(num) {  
    return num * num;  
}
```

```
let result = func(3);  
result = result + 1;  
alert(result); // выведет 10
```

Можно сразу выполнять какие-то действия с результатом работы функции перед записью в переменную:

```
function func(num) {  
    return num * num;  
}
```

```
let result = func(3) + 1;  
alert(result); // выведет 10
```

А можно не записывать результат в переменную, а сразу вывести его на экран:

```
function func(num) {  
    return num * num;  
}
```

```
alert(func(3)); // выведет 9
```

Использование функций в выражении

В следующем примере с помощью функции **func** мы сначала найдем квадрат числа 2, а затем - 3 квадрат числа, сложим эти значения и запишем в переменную:

```
function func(num) {  
    return num * num;  
}  
  
let result = func(2) + func(3);  
alert(result); // выведет 13
```

Функции в функциях

Можно также результат работы одной функции передать параметром в другую, например, вот так мы сначала найдем квадрат числа 2, а затем квадрат результата:

```
function func(num) {  
    return num * num;  
}  
  
let result = func(func(2));  
alert(result); // выведет 16
```

Функции, конечно же, не обязательно должны быть одинаковыми.

Пусть, например, у нас есть функция, возвращающая квадрат числа, и функция, возвращающая куб числа:

```
function square(num) {  
    return num * num;  
}  
  
function cube(num) {  
    return num * num * num;  
}
```



```
}
```

Давайте с помощью этих функций возведем число 2 в квадрат, а затем результат этой операции возведем в куб:

```
let result = cube(square(2));  
alert(result);
```

Пусть теперь у нас есть функция, возвращающая квадрат числа, и функция, находящая сумму двух чисел:

```
function square(num) {  
    return num * num;  
}
```

```
function sum(num1, num2) {  
    return num1 + num2;  
}
```

Найдем с помощью этих функций сумму квадрата числа 2 и сумму квадрата числа 3:

```
let result = sum(square(2), square(3));  
alert(result);
```

Тонкое место return

После того, как выполнится инструкция **return** - функция закончит свою работу. То есть: *после выполнения return больше никакой код не выполнится.*

Смотрите пример:

```
function func(num) {  
    return num * num;  
  
    alert('!'); // этот код никогда не выполнится  
}
```

```
let result = func(3);
```

Это не значит, что в функции должен быть **один return**. Но выполнится только один из них.

В следующем примере в зависимости от значения параметра выполнится либо первый, либо второй return:

```
function func(num) {  
    if (num >= 0) {  
        return '+';  
    } else {  
        return '-';  
    }  
}  
  
alert(func( 3)); // выведет '+'  
alert(func(-3)); // выведет '-'
```

Цикл и return

Пусть у нас есть функция, возвращающая сумму чисел от 1 до 5:

```
function func() {  
    let sum = 0;  
  
    for (let i = 1; i <= 5; i++) {  
        sum += i;  
    }  
  
    return sum;  
}
```

```
let result = func();  
alert(result); // выведет 15
```

Обратите внимание на то, что **return** расположен после цикла.

Пусть теперь мы расположим **return** внутри цикла, вот так:

```
function func() {  
    let sum = 0;  
  
    for (let i = 1; i <= 5; i++) {  
        sum += i;  
        return sum;  
    }  
}
```

```
let result = func();  
alert(result);
```

В этом случае цикл прокрутится лишь одну итерацию и произойдет автоматический выход из функции (ну и заодно из цикла).

А за одну итерацию цикла в переменной **sum** окажется лишь число 1, а не вся нужная сумма.

Применение return в цикле

То, что **return** расположен внутри цикла, не всегда может быть ошибкой.

В следующем примере сделана функция, которая определяет, сколько первых элементов массива нужно сложить, чтобы сумма стала больше или равна 10:

```
function func(arr) {  
    let sum = 0;  
  
    for (let i = 0; i < arr.length; i++) {  
        sum += arr[i];  
  
        // Если сумма больше или равна 10:  
        if (sum >= 10) {  
            return i + 1; // выходим из цикла и из функции  
        }  
    }  
}
```

```
let result = func([1, 2, 3, 4, 5]);  
alert(result);
```

А в следующем примере сделана функция, которая вычисляет, сколько целых чисел, начиная с 1, нужно сложить, чтобы результат был больше 100:

```
function func() {  
    let sum = 0;  
    let i = 1;  
  
    while (true) { // бесконечный цикл  
        sum += i;  
  
        if (sum >= 100) {  
            return i; // цикл крутится пока не выйдет тут  
        }  
    }  
}
```

```
    }  
  
    i++;  
}  
}
```

```
alert( func() );
```

Примеры на создание функций в JavaScript

В этом уроке мы с вами будем отрабатывать полученные ранее знания по пользовательским функциям на практических задачах.

Пусть у нас дан какой-то произвольный массив с числами, например, такой:

```
let arr = [1, 2, 3, 4];
```

Пусть теперь перед нами, к примеру, стоит задача найти сумму элементов этого массива.

Пока не будем использовать пользовательские функции, чтобы показать, какие проблемы возникнут в этом случае.

Давайте напишем код, реализующий нашу задачу:

```
let arr = [1, 2, 3, 4];
```

```
// Код, находящий сумму:
```

```
let sum = 0;
```

```
for (let elem of arr) {  
    sum += elem;  
}
```

```
alert(sum);
```

Пусть теперь у нас даны два массива:

```
let arr1 = [1, 2, 3, 4];
```

```
let arr2 = [5, 6, 7, 8];
```

Пусть теперь перед нами стоит задача найти сумму элементов и одного массива, и второго. В этом случае нам придется код, находящий сумму, повторить два раза: для первого и для второго массива, вот так:

```
let arr1 = [1, 2, 3, 4];
let arr2 = [5, 6, 7, 8];

// Код, находящий сумму первого массива:
let sum1 = 0;

for (let elem of arr1) {
    sum1 += elem;
}

// Код, находящий сумму второго массива:
let sum2 = 0;

for (let elem of arr2) {
    sum2 += elem;
}

// Код, выводящий найденные суммы:
alert(sum1);
alert(sum2);
```

Как мы видим, у нас возникает дублирование кода. Давайте избавимся от него, создав свою функцию, находящую сумму элементов переданного параметром массива.

Вот код этой функции:

```
function getSum(arr) {
    let sum = 0;

    for (let elem of arr) {
        sum += elem;
    }

    return sum;
}
```

Используем теперь нашу функцию для нахождения суммы одного и второго массива:

```
function getSum(arr) {
    let sum = 0;

    for (let elem of arr) {
        sum += elem;
    }
}
```

```
    return sum;
}
```

```
let arr1 = [1, 2, 3, 4];
let arr2 = [5, 6, 7, 8];
alert(getSum(arr1));
alert(getSum(arr2));
```

Решение задач

- 1 Сделайте функцию, выводящую на экран ваше имя.
- 2 Сделайте функцию, выводящую на экран сумму чисел от 1 до 100.
- 3 Сделайте функцию, которая параметром принимает число, а возвращает куб этого числа. С помощью этой функции найдите куб числа 3 и запишите его в переменную **result**.
- 4 Сделайте функцию, которая параметром принимает число, а возвращает квадратный корень из этого числа.

С помощью этой функции найдите корень числа 3, затем найдите корень числа 4. Просуммируйте полученные результаты и выведите их на экран.

- 5 Пусть у вас есть функция, возвращающая квадратный корень из числа, и функция, округляющая дробь до трех знаков в дробной части:

```
function sqrt(num) {
    return Math.sqrt(num);
}
```

```
function round(num) {
    return num.toFixed(3);
}
```

С помощью этих функций найдите квадратный корень из числа 2 и округлите его до трех знаков в дробной части.

6 Пусть у вас есть функция, возвращающая квадратный корень из числа, и функция, возвращающая сумму трех чисел:

```
function sqrt(num) {  
    return Math.sqrt(num);  
}  
  
function sum(num1, num2, num3) {  
    return num1 + num2 + num3;  
}
```

С помощью этих функций найдите сумму корней чисел 2, 3 и 4 и запишите ее в переменную **result**.

7. Напишите функцию, которая параметром будет принимать число и делить его на 2 столько раз, пока результат не станет меньше 10. Пусть функция возвращает количество итераций, которое потребовалось для достижения результата.

8 Напишите функцию, которая будет находить сумму квадратов элементов массива.

9 Реализуйте функцию **getDel**, которая параметром будет принимать число и возвращать массив его делителей, то есть чисел, на которое делится наше число. К примеру, если мы передадим число **24** - мы должны получить массив [1, 2, 3, 4, 6, 8, 12, 24].